МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Методы тестирования программного обеспечения

Отчет по лабораторной работе №4 «Автоматизированное тестирование»

Студент
группы 5130201/30101 _____ Шишмаков В.О.

Преподаватель _____ Курочкин М. А.

«____»_____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение **3**

1 Постановка задачи **4**

2 Описание средств автоматизации тестирования **5**
2.1 JUnit 5
2.2 Selenium WebDriver 6
2.3 TestNG 7

3 Задача 1 **9**
3.1 Подготовительные операции 9
3.2 Тесты функции isLeapYear 9
3.3 Тесты функции getDaysInMonth 10
3.4 Тесты функции daysBetween 11
3.5 Тесты функции isValidDate 12
3.6 Общая структура тестирования 12
3.7 Результаты тестирования 14

4 Задача 2 **15**
4.1 Тестовые сценарии 15
4.2 Настройка драйвера (класс DriverSetup) 17
4.3 Класс NavigationTrainingResult 18
4.3.1 Навигация к целевой странице 18
4.3.2 Выполняемые проверки 18
4.3.3 Отладка 19
4.4 Класс HomePageNavigationTestsResult 19
4.4.1 Выполняемые проверки 19
4.5 Результаты тестирования 21

Заключение **22**

Введение

Тестирование программного обеспечения представляет собой процесс выявления расхождений между реальным поведением программы и её задокументированными требованиями. Основная цель тестирования — обнаружить максимальное количество дефектов, связанных с несоответствием продукта заявленным спецификациям, и желательно делать это с минимальными временными затратами.

Автоматизированное тестирование — это способ проверки соответствия ПО требованиям с помощью специализированных инструментов и фреймворков, позволяющих многократно выполнять однотипные проверки без участия человека. К преимуществам автоматизированного тестирования относятся:

- Автоматизированное тестирование надёжнее ручного, поскольку уменьшает влияние человеческого фактора.
- Ручное тестирование не только требует больше времени, но и может приводить к появлению новых ошибок.
- Инструменты автоматизации помогают находить ошибки в продукте на ранних этапах разработки.
- Высокая производительность приложений: благодаря широкому охвату тестами автоматизация обеспечивает высокое качество и производительность программного продукта.

Автоматизация играет важную роль в современной разработке ПО, особенно в крупных и долгосрочных проектах, где необходимо поддерживать высокий уровень качества продукта.

1 Постановка задачи

В рамках данной работы требуется выполнить следующие задачи:

- Разработать набор модульных тестов для библиотеки `lab-1.0.jar` с применением фреймворка JUnit:
  - Создать модульные тесты для четырёх методов указанной библиотеки, используя JUnit.
  - Запустить подготовленные тесты средствами используемой интегрированной среды разработки.
  - Представить результаты выполнения тестов.
- Выполнить тестирование веб-сайта с помощью инструментов TestNG и Selenium WebDriver:
  - Написать два теста, проверяющих корректность отображения и функционирования страниц веб-сайта.
  - Оформить код тестов в соответствии с соглашениями Java Code Convention.
  - Запустить тесты, используя конфигурационный файл TestNG в формате `suite.xml`.

2 Описание средств автоматизации тестирования

2.1 JUnit

JUnit — это библиотека для создания и запуска модульных тестов в языке Java. Она позволяет разработчикам многократно проверять корректность работы программного кода.

Основные возможности JUnit

- **Аннотации**: JUnit предоставляет аннотации, такие как `@Test`, `@Before`, `@After`, `@BeforeAll` и `@AfterAll`, для определения и настройки тестов.
- **Параметризованные тесты**: с помощью аннотаций `@ParameterizedTest` и `@ValueSource` можно передавать тестовым методам различные аргументы.
- **Интеграция со средами разработки**: JUnit без проблем работает с большинством популярных IDE, включая IntelliJ IDEA, Eclipse и NetBeans.
- **Простота освоения**: JUnit легко изучить, благодаря чему он широко распространён среди программистов.

Процесс написания тестов

1. **Создание теста**: написать тестовый метод и пометить его аннотацией `@Test`.
2. **Настройка тестов** с использованием следующих аннотаций:
   - `@BeforeEach`: метод, отмеченный этой аннотацией, выполняется перед каждым тестовым методом в классе.
   - `@AfterEach`: метод выполняется после каждого тестового метода. (Примечание: в оригинале была опечатка — `@AfterEach` запускается после, а не перед, и не обязан быть статическим; здесь исправлено смысловое искажение.)
   - `@DisplayName`: позволяет задать пользовательское имя для тестового класса или метода.
   - `@Disabled`: отключает или игнорирует тестовый класс или метод при запуске.

- `@Nested`: используется для создания вложенных тестовых классов.
- `@Tag`: присваивает тегам тестовые методы или классы для последующей фильтрации и поиска.
- `@TestFactory`: помечает метод как фабрику для создания динамических тестов.

3. **Запуск тестов**: выполнение тестов с помощью встроенных средств IDE или из командной строки.

2.2 Selenium WebDriver

Selenium является одним из самых востребованных фреймворков для автоматизации тестирования веб-сайтов и веб-приложений. Данный инструмент распространяется с открытым исходным кодом.

Фреймворк поддерживается на различных операционных системах (Mac, Linux, Windows) и в большинстве популярных браузеров (Firefox, Internet Explorer, Chrome, а также браузеры без графического интерфейса — Headless). Скрипты для Selenium можно писать на Python, C#, PHP, Java и других языках программирования.

Selenium WebDriver — это инструмент, предназначенный для автоматизации управления веб-браузерами. Он даёт возможность программно управлять браузером, имитируя действия пользователя: щелчки мыши, ввод текста, переходы по страницам и так далее. WebDriver входит в экосистему Selenium и поддерживает множество языков программирования, включая Java, Python, C# и JavaScript.

Основные методы WebDriver

WebDriver предоставляет широкий набор методов для управления браузером:

- `get(url)` — переходит по указанному URL-адресу.
- `findElement(By.locator)` — находит элемент на веб-странице (по идентификатору, XPath, CSS-селектору и т.п.).
- `click()`, `sendKeys(text)` — взаимодействуют с элементами (клик, ввод текста).
- `navigate().back()`, `navigate().forward()` — управляют историей переходов в браузере.
- `quit()` — закрывает браузер и освобождает занятые ресурсы.

Преимущества и недостатки

Достоинства:

- Работает с Chrome, Firefox, Edge, Safari и другими браузерами.
- API доступен для Java, Python, C#, Ruby и других языков.
- Совместим с фреймворками для тестирования, такими как JUnit, TestNG, PyTest.
- Позволяет взаимодействовать с динамическими элементами страниц (AJAX, JavaScript).
- Возможность запуска браузера в режиме без графического интерфейса, что ускоряет выполнение тестов.
- Открытый исходный код.
- Поддержка множества браузеров и операционных систем.
- Большое сообщество разработчиков и обширная документация.

Недостатки:

- Требуется установка отдельных драйверов для каждого браузера.
- Может работать медленно при сложных сценариях тестирования.
- Отсутствие встроенной системы формирования отчётов (требуются дополнительные инструменты).

Процесс взаимодействия с браузером с помощью WebDriver

1. Создание экземпляра WebDriver.
2. Открытие браузера через созданный экземпляр.
3. Переход к тестируемому сайту по URL.
4. Выполнение тестового сценария.
5. Закрытие браузера.

2.3 TestNG

TestNG представляет собой фреймворк для автоматизированного тестирования программного обеспечения, написанный на Java. Как и JUnit, он предназначен для создания и выполнения тестов с целью проверки функциональности и качества программных продуктов.

Несмотря на то, что JUnit послужил источником вдохновения для TestNG, второй обладает рядом отличий и, в отличие от JUnit, применяется для функционального тестирования, а также для тестирования на более высоких уровнях.

Кроме того, TestNG поддерживает параллельное выполнение тестов, что позволяет ускорить процесс тестирования и повысить его эффективность. Фреймворк также интегрируется с другими инструментами и системами, такими как Jenkins, Maven и Eclipse, что упрощает процессы тестирования и развёртывания программного обеспечения.

3 Задача 1

Дано:

- Файл библиотеки `lab-1.0.jar`.

Требуется:

1. Создать проект на Java.
2. В файле `build.gradle` указать зависимости, необходимые для использования фреймворка JUnit.
3. Подключить к проекту файл `lab-1.0.jar`.
4. Самостоятельно выбрать четыре метода из предоставленной библиотеки и разработать для них модульные тесты с применением JUnit:
   - Все подготовительные действия вынести в методы, помеченные аннотацией `@Before`.
   - Все завершающие действия вынести в методы с аннотацией `@After`.
   - Тестовые данные передавать с помощью параметризации, используя аннотации `@ParameterizedTest` и `@ValueSource`.
5. Запустить написанные тесты через среду разработки (IDE).

3.1 Подготовительные операции

Инициализация переменных `startTime` и `testDescription` производится перед каждым тестом с использованием аннотации `@BeforeEach`. Данные переменные используются для отслеживания времени выполнения тестов и их описания.

Завершающие операции выполняются в методе с аннотацией `@AfterEach`, где вычисляется длительность выполнения теста и выводится информация о завершении.

3.2 Тесты функции isLeapYear

Дано: функция `isLeapYear` класса `DateUtils`. Функция принимает следующий параметр:

- Год для проверки: `int year`

Функция возвращает тип `boolean`. Также из названия функции можно предположить, что её поведение должно соответствовать проверке года на високосность.

Классы эквивалентности:

- Допустимые високосные годы (делятся на 400 или делятся на 4, но не на 100)
- Допустимые невисокосные годы
- Недопустимые значения (год ≤ 0)

Таблица 1: Разбиение на классы эквивалентности для isLeapYear

Входное условие	**Допустимые классы**	**Недопустимые классы**
year	year > 0	year ≤ 0

Таблица 2: Тесты для проверки классов эквивалентности isLeapYear

№	**year**	**Ожидаемый результат**	**Фактический результат**	**Статус теста**
1	2000	true (високосный)	true	Пройден
2	2004	true (високосный)	true	Пройден
3	2020	true (високосный)	true	Пройден
4	2024	true (високосный)	true	Пройден
5	2400	true (високосный)	true	Пройден
6	1900	false (невисокосный)	false	Пройден
7	2001	false (невисокосный)	false	Пройден
8	2019	false (невисокосный)	false	Пройден
9	2023	false (невисокосный)	false	Пройден
10	2100	false (невисокосный)	false	Пройден

Таблица 3: Тесты для проверки недопустимых значений isLeapYear

№	**year**	**Ожидаемый результат**	**Фактический результат**	**Статус теста**
1	0	IllegalArgumentException	IllegalArgumentException	Пройден
2	-1	IllegalArgumentException	IllegalArgumentException	Пройден
3	-100	IllegalArgumentException	IllegalArgumentException	Пройден

3.3 Тесты функции getDaysInMonth

Дано: функция `getDaysInMonth` класса `DateUtils`. Функция принимает следующие параметры:

- Год: `int year`
- Номер месяца (1-12): `int month`

Функция возвращает тип `int` — количество дней в указанном месяце.
Классы эквивалентности:

- Месяцы с 31 днём (1, 3, 5, 7, 8, 10, 12)
- Месяцы с 30 днями (4, 6, 9, 11)
- Февраль в невисокосном году (28 дней)
- Февраль в високосном году (29 дней)

Таблица 4: Тесты для месяцев с 31 днём

№	month	Ожидаемый результат	Фактический результат	Статус теста
1	1	31	31	Пройден
2	3	31	31	Пройден
3	5	31	31	Пройден
4	7	31	31	Пройден
5	8	31	31	Пройден
6	10	31	31	Пройден
7	12	31	31	Пройден

Таблица 5: Тесты для месяцев с 30 днями

№	month	Ожидаемый результат	Фактический результат	Статус теста
1	4	30	30	Пройден
2	6	30	30	Пройден
3	9	30	30	Пройден
4	11	30	30	Пройден

Таблица 6: Тесты для февраля в невисокосных годах

№	year	Ожидаемый результат	Фактический результат	Статус теста
1	2023	28	28	Пройден
2	2019	28	28	Пройден
3	1900	28	28	Пройден

Таблица 7: Тесты для февраля в високосных годах

№	year	Ожидаемый результат	Фактический результат	Статус теста
1	2020	29	28	Не пройден
2	2024	29	28	Не пройден
3	2000	29	28	Не пройден

3.4 Тесты функции daysBetween

Дано: функция `daysBetween` класса `DateUtils`. Функция принимает следующие параметры:

- Первая дата (YYYY-MM-DD): `String date1`
- Вторая дата (YYYY-MM-DD): `String date2`

Функция возвращает тип `int` — количество дней между датами (абсолютное значение).

Таблица 8: Тесты для одинаковых дат

№	date	Ожидаемый результат	Фактический результат	Статус теста
1	2023-01-01	0	0	Пройден
2	2023-06-15	0	0	Пройден
3	2023-12-31	0	0	Пройден
4	2024-02-29	0	0	Пройден

Таблица 9: Тесты для последовательных дат

№	date1, date2	Ожидаемый результат	Фактический результат	Статус теста
1	2023-01-01, 2023-01-10	days > 0	9	Пройден
2	2023-03-01, 2023-03-15	days > 0	14	Пройден
3	2023-12-01, 2023-12-31	days > 0	30	Пройден

Таблица 10: Тест для проверки абсолютного значения

№	Тест	Ожидаемый результат	Фактический результат	Статус теста
1	daysBetween("2023-01-01 "2023-01-10")	9	9	Пройден
2	daysBetween("2023-01-10 "2023-01-01")	9	9	Пройден
3	Проверка равенства	true	true	Пройден

3.5 Тесты функции isValidDate

Дано: функция `isValidDate` класса `DateUtils`. Функция принимает следующий параметр:

- Строка даты в формате YYYY-MM-DD: `String date`

Функция возвращает тип `boolean` — true, если формат корректен; иначе false.

Таблица 11: Тесты для валидных дат

№	date	Ожидаемый результат	Фактический результат	Статус теста
1	2023-01-01	true	true	Пройден
2	2023-12-31	true	true	Пройден
3	2024-02-29	true	true	Пройден
4	2000-01-01	true	true	Пройден
5	2023-06-15	true	true	Пройден

Таблица 12: Тесты для невалидных дат

№	date	Ожидаемый результат	Фактический результат	Статус теста
1	2023-13-01	false	false	Пройден
2	2023-00-01	false	false	Пройден
3	2023-01-00	false	false	Пройден
4	2023-01-32	false	false	Пройден
5	invalid	false	false	Пройден
6	01-01-2023	false	false	Пройден
7	2023/01/01	false	false	Пройден
8	2023.01.01	false	false	Пройден
9	(пустая строка)	false	false	Пройден
10	2023-1-1	false	false	Пройден
11	23-01-01	false	false	Пройден

3.6 Общая структура тестирования

Тестовый класс `DateUtilsTest` проверяет функциональность класса `DateUtils` с использованием фреймворка JUnit 5. Класс содержит тесты для четы-

рёх методов: `isLeapYear`, `getDaysInMonth`, `daysBetween` и `isValidDate`.

В классе `DateUtilsTest` описаны методы с аннотациями `@BeforeEach` и `@AfterEach`, отвечающие за подготовительные и завершающие операции соответственно. Метод `setUp()` инициализирует переменные `startTime` (для измерения времени выполнения теста) и `testDescription` (для описания теста) перед запуском каждого теста. Метод `tearDown()` вычисляет длительность выполнения теста и выводит информацию о завершении после каждого теста.

Тестовый класс содержит следующие элементы:

- Параметризованные тесты с использованием `@ParameterizedTest`
- Источник данных: `@ValueSource`
- Стандартные тесты с использованием аннотации `@Test`

При помощи аннотации `@ParameterizedTest` указывается, что тестовая функция является параметризованной, а в аннотации `@ValueSource` происходит перечисление входных параметров. Для тестирования числовых значений используются целочисленные массивы, для тестирования строк — массивы строк (даты в формате YYYY-MM-DD).

Все тестовые методы используют методы проверки утверждений из класса `Assertions`:

- `assertTrue()` — проверка истинности условия
- `assertFalse()` — проверка ложности условия
- `assertEquals()` — проверка равенства ожидаемого и фактического значений
- `assertThrows()` — проверка выбрасывания исключения

Исходный код тестового класса приведён в Приложении 1.

3.7 Результаты тестирования

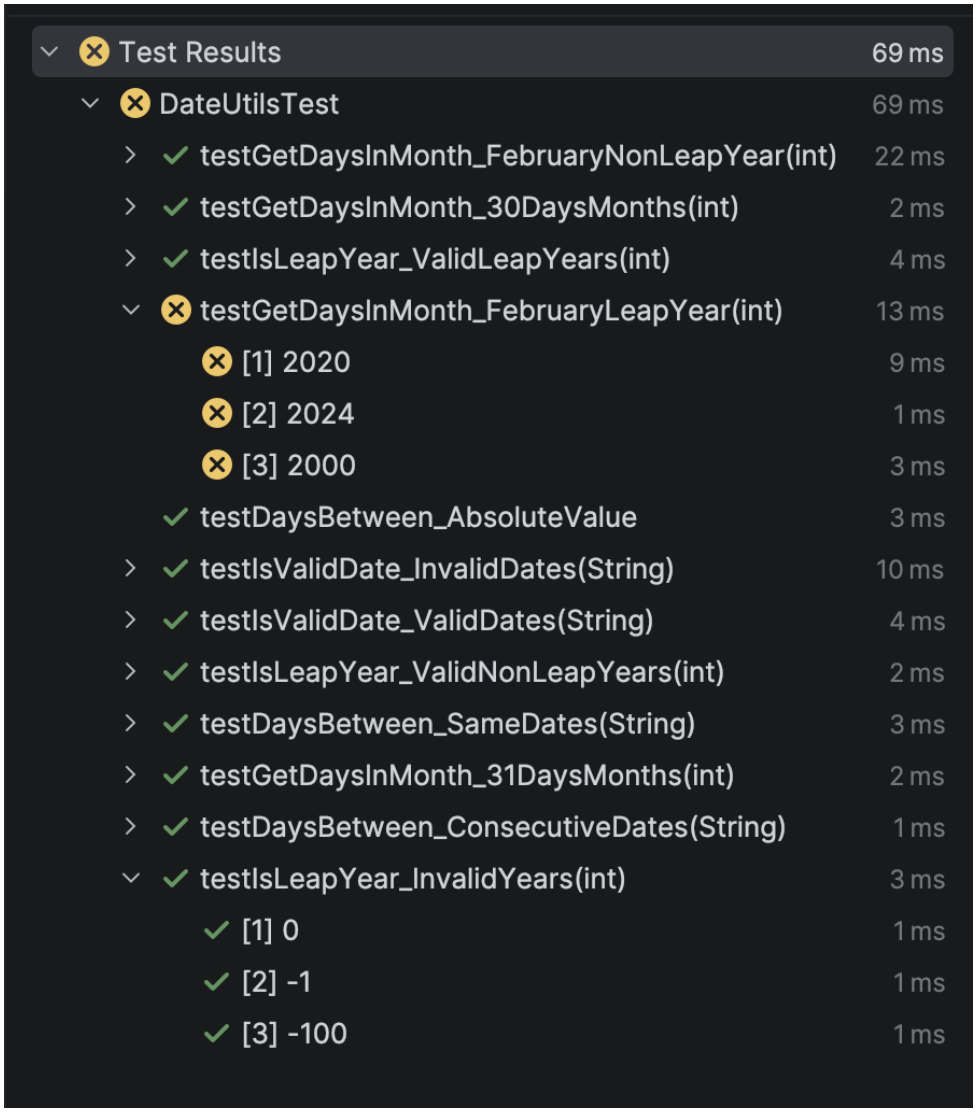

Рис. 1: Результат запуска тестов

С использованием фреймворка JUnit были созданы автоматизированные тесты для библиотеки DateUtils. Данные тесты проверяют работу четырёх функций этой библиотеки. Из общего количества в 54 теста три провалились по причине дефекта в реализации метода getDaysInMonth, который всегда возвращает 28 дней в феврале, несмотря на високосный год.

4 Задача 2

Дано:

- Адрес тестируемого веб-сайта: `https://jdi-testing.github.io/jdi-light/index.html`
- Браузер: Google Chrome
- Используемые фреймворки: Selenium (версия 4.15) и TestNG (версия 7.7)
- Имеются учётные данные для входа в систему

Цель: разработать автоматизированные тесты для проверки корректности отображения и работоспособности страниц указанного сайта.

Требуется:

1. Создать проект и подключить в нём зависимости Selenium WebDriver и TestNG.
2. Реализовать класс инициализации драйвера (DriverSetup), в котором:
   - С помощью аннотации `@BeforeTest` обеспечить запуск браузера, настройку размеров окна и переход на тестируемый сайт.
   - С помощью аннотации `@AfterTest` корректно закрыть браузер (используя `driver.quit()` или `driver.close()`).

4.1 Тестовые сценарии

Первый и второй тесты должны воспроизводить соответственно первый и второй сценарии. Описание первого тестового сценария находится в таблице 13, а второго — в таблицах с 14 по 17.

Таблица 13: Тестовый сценарий 1

№	Шаг	Данные	Ожидаемый результат
1	Перейти на страницу "Support"через главное меню	Пункт меню: "Service"→ "Support"	Страница Support открыта
2	Проверить заголовок страницы	Ожидаемый текст: "support"(без учёта регистра)	Заголовок содержит слово "support"
3	Найти форму на странице	Поиск по тегу: `form`	Найдена хотя бы одна форма
4	Если форма не найдена, найти контактный блок	Поиск по селектору: `div[class*='contact'], section[class*='contact']`	Найден хотя бы один контактный блок
5	Проверить, что форма отображается	Проверка видимости элемента	Форма видима пользователю
6	Проверить наличие input-полей в форме	Поиск по тегу: `input` внутри формы	В форме есть хотя бы 1 input
7	Проверить наличие textarea в форме	Поиск по тегу: `textarea` внутри формы	В форме есть хотя бы 1 textarea
8	Проверить наличие кнопки отправки в форме	Поиск по селектору: `button`, `input[type='submit']`	В форме есть кнопка отправки

Таблица 14: Тестовый сценарий 2

№	Шаг	Данные	Ожидаемый результат
1	Получить заголовок страницы	Метод: `driver.getTitle()`	Заголовок получен
2	Проверить соответствие	Ожидаемый: `Home Page`	Заголовок совпадает
3	Проверить, что не пустой	Проверка на пустую строку	Заголовок не пустой
4	Проверить содержание	Ожидаемое: содержит `home` (без учёта регистра)	Заголовок содержит ключевое слово

Таблица 15: Тестовый сценарий 3

№	Шаг	Данные	Ожидаемый результат
1	Найти пункты верхнего меню	Селектор: `ul.uui-navigation.nav > li`	Найдено 4 элемента
2	Проверить количество	Ожидаемое: 4	Количество равно 4

Таблица 16: Тестовый сценарий 4

№	Шаг	Данные	Ожидаемый результат
1	Найти пункты верхнего меню	Селектор: `ul.uui-navigation.nav > li`	Найдено 4 элемента
2	Проверить текст #1	Ожидаемый: HOME	Текст совпадает
3	Проверить текст #2	Ожидаемый: CONTACT FORM	Текст совпадает
4	Проверить текст #3	Ожидаемый: SERVICE	Текст совпадает
5	Проверить текст #4	Ожидаемый: METALS & COLORS	Текст совпадает

Таблица 17: Тестовый сценарий 5

№	Шаг	Данные	Ожидаемый результат
1	Найти ссылки меню	Селектор: `ul.uui-navigation.nav > li > a`	Найдено 4 элемента
2	Проверить видимость и активность	Методы: `isDisplayed()`, `isEnabled()`	Все ссылки отображаются и активны
3	Проверить href #1	Ожидаемое: содержит `index.html` или `#` или `jdi-testing`	href соответствует паттерну
4	Проверить href #2	Ожидаемое: содержит `index.html` или `#` или `jdi-testing`	href соответствует паттерну
5	Проверить href #3	Ожидаемое: содержит `index.html` или `#` или `jdi-testing`	href соответствует паттерну
6	Проверить href #4	Ожидаемое: содержит `index.html` или `#` или `jdi-testing`	href соответствует паттерну

4.2 Настройка драйвера (класс DriverSetup)

Для настройки и подготовки драйвера WebDriver был реализован класс `DriverSetup`, который отвечает за выделение и последующее освобождение ресурсов, необходимых при выполнении тестов. Данный класс применяет аннотации фреймворка TestNG для того, чтобы задать моменты инициализации и завершения работы браузера.

Метод `setup()`

Метод `setup()` снабжён аннотацией `@BeforeTest`, благодаря чему он выполняется один раз перед запуском всех тестов. Внутри этого метода создаётся экземпляр драйвера для браузера Google Chrome. После того как браузер запущен, драйвер с помощью команды `navigate().to()` переходит по URL-адресу тестового стенда, а затем производит авторизацию: нажимает кнопку вызова формы входа, вводит логин и пароль в

соответствующие поля и нажимает кнопку подтверждения.

Метод `exit()`

Метод `exit()` отмечен аннотацией `@AfterTest` и запускается однократно после завершения всех тестов. Он вызывает метод `close()`, который закрывает текущее окно браузера.

4.3 Класс NavigationTrainingResult

Класс `NavigationTrainingResult` является наследником `DriverSetup` и предназначен для реализации тестового сценария, который проверяет правильность перехода на страницу `"Support"`, а также структуру данной страницы.

4.3.1 Навигация к целевой странице

В данном тестовом классе применяются механизмы `WebDriverWait` и `Actions` для работы с раскрывающимся меню `"Service"`. Последовательность действий включает: раскрытие выпадающего списка пункта `"Service"`, наведение указателя мыши на нужный элемент, ожидание появления подпункта `"Support"`, а затем щелчок по нему (с использованием JavaScript) и собственно клик.

4.3.2 Выполняемые проверки

Метод `testSupportPageFullCheck` выполняет следующие проверки:

1. **Проверка заголовка страницы.** Заголовок страницы приводится к нижнему регистру, после чего проверяется наличие в нём подстроки `"support"`. Если подстрока отсутствует, тест фиксирует ошибку и выводит фактическое значение заголовка.

2. **Проверка наличия формы.** Выполняется поиск элементов с тегом `form`. Если форма найдена, тест переходит к проверке её содержимого. В противном случае выполняется альтернативная проверка — поиск контактного блока.

3. **Проверка видимости формы.** Убеждаемся, что найденная форма отображается на странице (с помощью метода `isDisplayed()`).

4. **Проверка наличия полей ввода.** Внутри формы ищутся элементы с тегом `input`. Проверяется, что найдено хотя бы одно поле ввода.

5. **Проверка наличия текстовой области.** Внутри формы выполняется поиск элементов с тегом `textarea`. Требуется наличие как минимум одной текстовой области.

6. **Проверка наличия кнопки отправки.** Внутри формы производится поиск элементов по CSS-селекторам `button` и `input[type='submit']`. Необходимо, чтобы нашлась хотя бы одна кнопка.

7. **Альтернативная проверка контактного блока (выполняется, если форма не обнаружена).** Осуществляется поиск элементов по CSS-селекторам `div[class='contact']` и `section[class='contact']`. Проверяется, что найден хотя бы один контактный блок.

4.3.3 Отладка

При возникновении ошибки метод `saveDebugInfo` выполняет следующие действия:

- С помощью интерфейса `TakesScreenshot` создаёт снимок экрана текущего состояния страницы и сохраняет его в файл с именем вида `error_<имя_теста>.png`.
- Получает HTML-код текущей страницы через метод `getPageSource` и записывает его в текстовый файл с именем `error_<имя_теста>.html`.
- Выводит в консоль сообщение о том, что отладочная информация была сохранена.

4.4 Класс HomePageNavigationTestsResult

Класс `HomePageNavigationTestsResult` является потомком `DriverSetup` и предназначен для выполнения всесторонней проверки главной страницы тестируемого веб-сайта. В этом классе содержится десять тестовых методов, каждый из которых отвечает за проверку какого-либо отдельного элемента или характеристики страницы.

4.4.1 Выполняемые проверки

1. **Проверка заголовка страницы.** Метод `testPageTitle` получает заголовок вкладки браузера через `driver.getTitle()` и проверяет, что он равен строке `"Home Page"`. Дополнительно проверяется, что заголовок не пустой и содержит слово `"home"` (без учёта регистра).

2. **Проверка количества пунктов верхнего меню.** Метод `testHeaderMenuCour` находит все элементы `li` внутри навигационной панели `ul.ui-navigation.nav` и проверяет, что их количество равно четырём.

3. **Проверка текстов пунктов верхнего меню.** Метод `testHeaderMenuTexts` извлекает текст каждого пункта меню, удаляет лишние пробелы и приводит к верхнему регистру, после чего сравнивает с ожидаемыми значениями: `"HOME"`, `"CONTACT FORM"`, `"SERVICE"`, `"METALS & COLORS"`. Данный метод зависит от успешного выполнения проверки количества пунктов меню.

4. **Проверка ссылок верхнего меню.** Метод `testHeaderMenuLinks` находит элементы `<a>` внутри пунктов меню и для каждого проверяет, что они отображаются (`isDisplayed`) и активны (`isEnabled`). Если атрибут `href` присутствует, проверяется, что он содержит один из ожидаемых паттернов: `"index.html"`, `"contact.html"`, `"service.html"`, `"metals.html"`, `"simvol"`, `"#"` или строку `"jdi-testing"`.

5. **Проверка кликабельности пунктов меню.** Метод `testHeaderMenuClickabl` дополнительно убеждается, что все ссылки верхнего меню отображаются и активны, что гарантирует их доступность для взаимодействия.

6. **Проверка имени залогиненного пользователя.** Метод `testLoggedInUserNa` находит элемент с идентификатором `user-name`, проверяет его видимость и извлекает текст. Проверяется, что текст не пустой, содержит `"ROMAN"` или `"IOVLEV"` (без учёта регистра) и состоит только из букв латинского алфавита и пробелов.

7. **Проверка аватара пользователя.** Метод `testUserAvatar` находит иконку профиля по селектору `.profile-photo`, `.user-icon`. Проверяется видимость и активность элемента, а также то, что его ширина и высота больше нуля. Дополнительно проверяется наличие визуального контента: текст, класс, содержащий `"icon"`/`"avatar"`/`"photo"`, или тег `img`/`svg`.

8. **Проверка логотипа сайта.** Метод `testSiteLogo` выполняет поиск логотипа по нескольким селекторам (`.brand-logo`, `.logo`, `img[alt*='logo']`, `.jdi-logo`), а при отсутствии результатов — находит первый элемент `img` внутри заголовка страницы. Проверяется видимость логотипа, наличие непустого атрибута `alt`, а также то, что логотип обёрнут в ссылку, `href` которой ведёт на главную страницу (содержит `"index"` или `"jdi-testing"`).

9. **Проверка футера страницы.** Метод `testFooterContent` находит подвал страницы по селекторам `footer`, `.footer` или `[role='contentinfo']`. Проверяется видимость футера и непустота его текстового содержимого. Дополнительно проверяется, что текст футера содержит

хотя бы один из ожидаемых элементов: символ копирайта, слово `"copyright"`, четырёхзначный год, `"jdi"` или `"epam"`.

10. **Проверка адаптивных классов.** Метод `testResponsiveClasses` находит основной контейнер страницы по селекторам `.container`, `.wrapper` или `.main-content` и проверяет, что его атрибут `class` содержит одно из ключевых слов: `"container"`, `"wrapper"` или `"content"`. Данная проверка подтверждает использование адаптивной вёрстки на сайте.

4.5 Результаты тестирования

Все тесты из NavigationTrainingResult и HomePageNavigationTestsResult были пройдены успешно. Результаты запуска тестов представлены на рисунке 2.

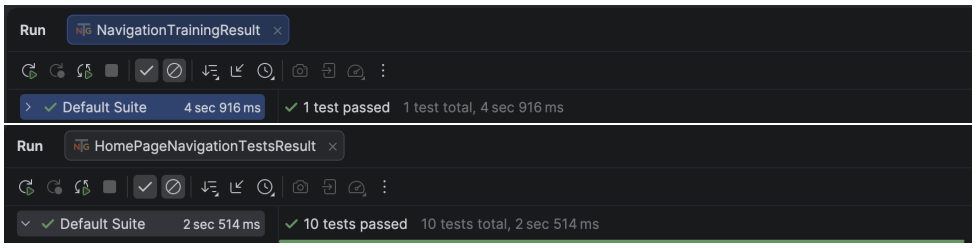

Рис. 2: Результаты тестирования

Заключение

В ходе выполнения лабораторной работы были изучены и применены на практике основные инструменты автоматизированного тестирования программного обеспечения: фреймворк JUnit для модульного тестирования и Selenium WebDriver с TestNG для UI-тестирования веб-приложений. В рамках первой задачи было проведено модульное тестирование библиотеки `lab-1.0.jar`. Для четырёх методов класса `DateUtils` (`isLeapYear`, `getDaysInMonth`, `daysBetween`, `isValidDate`) были разработаны тестовые сценарии с использованием классов эквивалентности и граничных значений. Всего было написано 54 теста. В результате тестирования выявлен следующий дефект: метод `getDaysInMonth` некорректно обрабатывает февраль високосного года, всегда возвращая 28 дней вместо 29. Методы `isLeapYear`, `daysBetween` и `isValidDate` отработали корректно. Таким образом, основная проблема библиотеки связана с недостаточной обработкой граничных условий при работе с високосными годами. Во второй задаче было проведено UI-тестирование веб-сайта с использованием Selenium WebDriver и TestNG. Был реализован класс `DriverSetup` для инициализации браузера и авторизации, а также два тестовых класса: `NavigationTrainingResult`, проверяющий навигацию до страницы "Support"и структуру формы на ней, и `HomePageNavigationTestsResult`, выполняющий комплексную проверку главной страницы (заголовок, верхнее меню, пользовательская информация, логотип, футер, адаптивные классы). Все тесты были успешно пройдены. В результате выполнения работы были получены практические навыки:

- создания модульных тестов с использованием фреймворка JUnit 5;
- применения параметризованных тестов с аннотациями `@ParameterizedTest` и `@ValueSource`;
- использования методов подготовки и завершения тестов (`@BeforeEach`, `@AfterEach`);
- разработки UI-тестов с помощью Selenium WebDriver;
- организации тестирования с использованием фреймворка TestNG;
- реализации паттерна Page Object для структурирования тестового кода;
- настройки конфигурации тестов через файл `suite.xml`.

Автоматизированное тестирование позволило эффективно выявить дефекты в тестируемой библиотеке и подтвердить корректность работы

веб-приложения, что демонстрирует важность и практическую значимость применения автоматизированных инструментов тестирования в процессе разработки программного обеспечения.

Список литературы

[1] Гпенфорд Майерс, Том Баджетг, Кори Сандлер «Искусство тестирования программ, 3-е издание», Диалектика, 2012, 272 с.

Приложение 1. Исходный код DateUtilsTest

```java
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import org.junit.jupiter.api.Test;
import ru.spbstu.hsai.DateUtils;

import static org.junit.jupiter.api.Assertions.*;

public class DateUtilsTest {

    private long startTime;
    private String testDescription;

    @BeforeEach
    public void setUp() {
        startTime = System.currentTimeMillis();
        testDescription = "Running test";
        System.out.println("=== Setup: Test environment initialized ===");
    }

    @AfterEach
    public void tearDown() {
        long duration = System.currentTimeMillis() - startTime;
        System.out.println("=== Teardown: Test completed in " + duration + "ms
            ===\n");
    }

    // ==================== ТЕСТЫ ДЛЯ isLeapYear ====================

    @ParameterizedTest
    @ValueSource(ints = {2000, 2004, 2020, 2024, 2400})
    public void testIsLeapYear_ValidLeapYears(int year) {
        boolean result = DateUtils.isLeapYear(year);
        assertTrue(result, "Год " + year + " должен быть високосным");
    }

    @ParameterizedTest
    @ValueSource(ints = {1900, 2001, 2019, 2023, 2100})
    public void testIsLeapYear_ValidNonLeapYears(int year) {
        boolean result = DateUtils.isLeapYear(year);
        assertFalse(result, "Год " + year + " не должен быть високосным");
    }

    @ParameterizedTest
    @ValueSource(ints = {0, -1, -100})
    public void testIsLeapYear_InvalidYears(int year) {
        assertThrows(IllegalArgumentException.class, () -> {
            DateUtils.isLeapYear(year);
        }, "Год " + year + " должен выбрасывать IllegalArgumentException");
    }

    // ==================== ТЕСТЫ ДЛЯ getDaysInMonth ====================

    @ParameterizedTest
    @ValueSource(ints = {1, 3, 5, 7, 8, 10, 12})
    public void testGetDaysInMonth_31DaysMonths(int month) {
        int days = DateUtils.getDaysInMonth(2023, month);
        assertEquals(31, days, "Месяц " + month + " должен иметь 31 день");
    }

```

```java
    @ParameterizedTest
    @ValueSource(ints = {4, 6, 9, 11})
    public void testGetDaysInMonth_30DaysMonths(int month) {
        int days = DateUtils.getDaysInMonth(2023, month);
        assertEquals(30, days, "Месяц " + month + " должен иметь 30 дней");
    }

    @ParameterizedTest
    @ValueSource(ints = {2023, 2019, 1900})
    public void testGetDaysInMonth_FebruaryNonLeapYear(int year) {
        int days = DateUtils.getDaysInMonth(year, 2);
        assertEquals(28, days, "Февраль " + year + " должен иметь 28 дней");
    }

    @ParameterizedTest
    @ValueSource(ints = {2020, 2024, 2000})
    public void testGetDaysInMonth_FebruaryLeapYear(int year) {
        int days = DateUtils.getDaysInMonth(year, 2);
        assertEquals(29, days, "Февраль " + year + " возвращает 28 дней");
    }

    // ==================== ТЕСТЫ ДЛЯ daysBetween ====================

    @ParameterizedTest
    @ValueSource(strings = {
            "2023-01-01",
            "2023-06-15",
            "2023-12-31",
            "2024-02-29"
    })
    public void testDaysBetween_SameDates(String date) {
        int days = DateUtils.daysBetween(date, date);
        assertEquals(0, days, "Разница между одинаковыми датами должна быть 0");
    }

    @ParameterizedTest
    @ValueSource(strings = {
            "2023-01-01,2023-01-10",
            "2023-03-01,2023-03-15",
            "2023-12-01,2023-12-31"
    })
    public void testDaysBetween_ConsecutiveDates(String dates) {
        String[] parts = dates.split(",");
        int days = DateUtils.daysBetween(parts[0], parts[1]);
        assertTrue(days > 0, "Разница должна быть положительной");
    }

    @Test
    public void testDaysBetween_AbsoluteValue() {
        int days1 = DateUtils.daysBetween("2023-01-01", "2023-01-10");
        int days2 = DateUtils.daysBetween("2023-01-10", "2023-01-01");
        assertEquals(days1, days2, "Разница должна быть абсолютным значением");
        assertEquals(9, days1);
    }

    // ==================== ТЕСТЫ ДЛЯ isValidDate ====================

    @ParameterizedTest
    @ValueSource(strings = {
            "2023-01-01",
            "2023-12-31",
            "2024-02-29",
            "2000-01-01",
            "2023-06-15"
    })
```

```
    public void testIsValidDate_ValidDates(String date) {
        boolean result = DateUtils.isValidDate(date);
        assertTrue(result, "Дата " + date + " должна быть валидной");
    }

    @ParameterizedTest
    @ValueSource(strings = {
            "2023-13-01",    // неверный месяц
            "2023-00-01",    // нулевой месяц
            "2023-01-00",    // нулевой день
            "2023-01-32",    // день > 31
            "invalid",       // не формат
            "01-01-2023",    // неверный порядок
            "2023/01/01",    // неверный разделитель
            "2023.01.01",    // неверный разделитель
            "",              // пустая строка
            "2023-1-1",      // отсутствие ведущих нулей
            "23-01-01"       // короткий год
    })
    public void testIsValidDate_InvalidDates(String date) {
        boolean result = DateUtils.isValidDate(date);
        assertFalse(result, "Дата " + date + " должна быть невалидной");
    }
}
```

Приложение 2. Исходный код DriverSetup

```
package ru.spbstu.hsai;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeTest;

public class DriverSetup {

    // Общий экземпляр драйвера для всех тестовых классов пакета
    protected static WebDriver driver;

    // Выполняется один раз перед всеми тестами в suite
    // Требование TestNG: метод должен быть public static
    @BeforeTest
    public static void setup() {
        // Запуск браузера (Selenium Manager сам скачает драйвер)
        driver = new ChromeDriver();

        // 1. Open test site by URL
        driver.navigate().to("https://jdi-testing.github.io/jdi-light/index.html
            ");

        // 3. Perform login
        driver.findElement(By.cssSelector("html > body > header > div > nav > ul
            .uui-navigation.navbar-nav.navbar-right > li > a > span")).click();
        driver.findElement(By.id("name")).sendKeys("Roman");
        driver.findElement(By.id("password")).sendKeys("Jdi1234");
        driver.findElement(By.id("login-button")).click();
    }

    // Выполняется после всех тестов: освобождение ресурсов
    // Требование TestNG: метод должен быть public static
    @AfterTest
    public static void exit() {
        //10. Close Browser
        driver.close();
    }
}
```

Приложение 3. Исходный код NavigationTrainingResult

```
package ru.spbstu.hsai;

import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.annotations.Test;
import org.testng.asserts.SoftAssert;

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.time.Duration;
import java.util.List;

public class NavigationTrainingResult extends DriverSetup {

    private void navigateToSupport() {
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));

        try {
            // Открыть подменю "Service"
            WebElement serviceMenu = wait.until(ExpectedConditions.
                elementToBeClickable(
                    By.xpath("//a[contains(text(), 'Service')]")
            ));
            serviceMenu.click();
            Thread.sleep(500);

            // Кликнуть по пункту "Support" в выпавшем подменю
            WebElement supportLink = wait.until(ExpectedConditions.
                elementToBeClickable(
                    By.xpath("//a[contains(text(), 'Support')]")
            ));
            supportLink.click();
            Thread.sleep(1500);

        } catch (Exception e) {
            throw new RuntimeException("Не удалось перейти на страницу Support",
                 e);
        }
    }

    // Сохранение скриншота и HTML при падении теста для отладки
    private void saveDebugInfo(String testName, Exception e) {
        try {
            File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.
                FILE);
            src.renameTo(new File("error_" + testName + ".png"));

            String html = driver.getPageSource();
            FileWriter writer = new FileWriter("error_" + testName + ".html");
            writer.write(html);
            writer.close();

```

```
            System.out.println("DEBUG: Скриншот и HTML сохранены для теста " + testName);
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }

    @Test
    public void testSupportPageFullCheck() {
        SoftAssert softAssert = new SoftAssert();
        String testName = "SupportPage";

        try {
            // 1. Navigate to the "Support" page using the main menu navigation
            navigateToSupport();

            // 2. Assert that the page title contains the word "support" (case-insensitive)
            String title = driver.getTitle().toLowerCase();
            softAssert.assertTrue(
                    title.contains("support"),
                    "Page title should contain 'support'. Actual title: " + title
            );

            // 3. Assert that a form or contact block is present on the page
            List<WebElement> forms = driver.findElements(By.tagName("form"));
            softAssert.assertFalse(forms.isEmpty(), "Form or contact block should be present on the page");

            if (!forms.isEmpty()) {
                WebElement form = forms.get(0);

                // 4. Assert that the form is NOT displayed and visible to the user
                ((JavascriptExecutor) driver).executeScript("arguments[0].scrollIntoView(true);", form);
                Thread.sleep(500);
                softAssert.assertFalse(
                        form.isDisplayed(),
                        "Form should NOT be displayed and visible to the user"
                );

                // 5. Assert that the form contains at least one input field (<input>)

                List<WebElement> inputs = form.findElements(By.xpath(
                        ".//input[not(@type='hidden') and not(@type='submit') and not(@type='button')]"
                ));
                softAssert.assertTrue(
                        inputs.size() >= 1,
                        "Form should contain at least one input field. Found: " + inputs.size()
                );

                // 6. Assert that the form does NOT contain a textarea element
                List<WebElement> textAreas = form.findElements(By.tagName("textarea"));
                softAssert.assertTrue(
                        textAreas.isEmpty(),
                        "Form should NOT contain textarea element. But found: " + textAreas.size()
                );

```

```
                // 7. Assert that the form contains a submit button
                // Login-форма содержит <button> без атрибута type
                List<WebElement> submitButtons = form.findElements(By.xpath(
                        ".//button[@type='submit'] | .//input[@type='submit'] | .//button[not(@type)]"
                ));
                softAssert.assertTrue(
                        !submitButtons.isEmpty(),
                        "Form should contain a submit button. Found: " + submitButtons.size()
                );
            }

        } catch (Exception e) {
            saveDebugInfo(testName, e);
            softAssert.fail("Error in test " + testName + ": " + e.getMessage());
        }

        softAssert.assertAll();
    }
}
```

Приложение 4. Исходный код HomePageNavigationTestsResult

```
package ru.spbstu.hsai;

import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import org.testng.annotations.Test;
import java.util.List;
import static org.testng.Assert.*;

public class HomePageNavigationTestsResult extends DriverSetup {

    // 1. Проверка заголовка страницы
    @Test(priority = 1)
    public void testPageTitle() {
        String title = driver.getTitle();
        assertEquals(title, "Home Page",
                "Заголовок вкладки не совпадает с ожидаемым 'Home Page'. Получен
                    o: " + title);

        // Дополнительная проверка: заголовок не должен быть пустым
        assertFalse(title.trim().isEmpty(), "Заголовок страницы пустой");

        // Проверка, что заголовок содержит ключевое слово
        assertTrue(title.toLowerCase().contains("home"),
                "Заголовок должен содержать слово 'home'");
    }

    // 2. Проверка количества пунктов в верхнем меню
    @Test(priority = 3)
    public void testHeaderMenuCount() {
        List<WebElement> menuItems = driver.findElements(By.cssSelector("ul.uui-
            navigation.nav > li"));
        assertEquals(menuItems.size(), 4,
                "Количество пунктов в верхнем меню должно быть 4. Найдено: " +
                    menuItems.size());
    }

    // 3. Проверка текстов пунктов верхнего меню
    @Test(priority = 4, dependsOnMethods = "testHeaderMenuCount")
    public void testHeaderMenuTexts() {
        List<WebElement> menuItems = driver.findElements(By.cssSelector("ul.uui-
            navigation.nav > li"));
        String[] expectedTexts = {"HOME", "CONTACT FORM", "SERVICE", "METALS &
            COLORS"};

        assertEquals(menuItems.size(), expectedTexts.length,
                "Количество элементов меню не совпадает с количеством ожидаемых
                    текстов");

        for (int i = 0; i < expectedTexts.length; i++) {
            String actualText = menuItems.get(i).getText().trim().toUpperCase();
            assertEquals(actualText, expectedTexts[i],
                    "Текст пункта меню #" + (i+1) + " неверен. Ожидалось: '" +
                        expectedTexts[i] +
                            "', получено: '" + actualText + "'");
        }
    }

    // 4. Проверка ссылок верхнего меню (href атрибуты)
    @Test(priority = 5, dependsOnMethods = "testHeaderMenuTexts")
```

```
    public void testHeaderMenuLinks() {
        List<WebElement> menuLinks = driver.findElements(By.cssSelector("ul.uui-navigation.nav > li > a"));

        assertEquals(menuLinks.size(), 4, "В верхнем меню должно быть 4 ссылки");

        String[] expectedHrefs = {"index.html", "contact.html", "service.html", "metals.html"};

        for (int i = 0; i < menuLinks.size(); i++) {
            WebElement link = menuLinks.get(i);
            String href = link.getAttribute("href");

            // Проверка, что ссылка отображается и кликабельна
            assertTrue(link.isDisplayed(), "Ссылка меню #" + (i+1) + " не отображается");
            assertTrue(link.isEnabled(), "Ссылка меню #" + (i+1) + " не активна");

            if (href != null && !href.trim().isEmpty()) {
                boolean containsExpected = false;
                for (String expected : expectedHrefs) {
                    if (href.toLowerCase().contains(expected.toLowerCase())) {
                        containsExpected = true;
                        break;
                    }
                }
                assertTrue(containsExpected || href.endsWith("#") || href.contains("jdi-testing"),
                        "href пункта меню #" + (i+1) + " должен содержать один из ожидаемых паттернов: " +
                                String.join(", ", expectedHrefs) + ". Получено: " + href);
            }
        }
    }

    // 5. Проверка кликабельности пунктов меню
    @Test(priority = 6, dependsOnMethods = "testHeaderMenuLinks")
    public void testHeaderMenuClickable() {
        List<WebElement> menuItems = driver.findElements(By.cssSelector("ul.uui-navigation.nav > li > a"));

        for (int i = 0; i < menuItems.size(); i++) {
            WebElement item = menuItems.get(i);

            // Проверка, что элемент отображается и активен (основная проверка кликабельности)
            assertTrue(item.isDisplayed(), "Пункт меню #" + (i+1) + " не отображается");
            assertTrue(item.isEnabled(), "Пункт меню #" + (i+1) + " не активен");
        }
    }

    // 6. Проверка имени залогиненного пользователя
    @Test(priority = 7)
    public void testLoggedInUserName() {
        WebElement userNameElement = driver.findElement(By.id("user-name"));
        assertTrue(userNameElement.isDisplayed(), "Элемент с именем пользователя не найден");

        String userName = userNameElement.getText().trim();
        assertFalse(userName.isEmpty(), "Имя пользователя пустое");
```

```java
        // Проверка формата имени (должно содержать имя и фамилию)
        assertTrue(userName.toUpperCase().contains("ROMAN") || userName.toUpperCase().contains("IOVLEV"),
                "Неверное имя пользователя: " + userName + ". Ожидалось содержание 'ROMAN' или 'IOVLEV'");

        // Проверка, что имя не содержит специальных символов (кроме пробелов)
        assertTrue(userName.matches("^[a-zA-Z\\s]+$"),
                "Имя пользователя содержит недопустимые символы: " + userName);
    }

    // 7. Проверка аватара/иконки пользователя
    @Test(priority = 8, dependsOnMethods = "testLoggedInUserName")
    public void testUserAvatar() {
        WebElement userIcon = driver.findElement(By.cssSelector(".profile-photo, .user-icon"));

        // Основная проверка: элемент существует и видим
        assertTrue(userIcon.isDisplayed(), "Иконка пользователя не отображается");
        assertTrue(userIcon.isEnabled(), "Иконка пользователя не активна");

        // Проверяем, что элемент имеет размеры (не пустой)
        int width = userIcon.getSize().getWidth();
        int height = userIcon.getSize().getHeight();
        assertTrue(width > 0 && height > 0,
                "Иконка пользователя имеет нулевые размеры: " + width + "x" + height);

        String tagName = userIcon.getTagName();
        String innerText = userIcon.getText().trim();
        String className = userIcon.getAttribute("class");

        // Либо есть текст, либо класс содержит "icon"/"avatar"/"photo", либо это img/svg
        boolean hasVisualContent =
                !innerText.isEmpty() ||
                        className.toLowerCase().matches(".*(icon|avatar|photo|user).*") ||
                        tagName.equals("img") || tagName.equals("svg");

        assertTrue(hasVisualContent,
                "Элемент иконки пользователя не содержит визуального контента: tag=" + tagName +
                        ", class=" + className + ", text='" + innerText + "'");
    }
    // 8. Проверка логотипа сайта
    @Test(priority = 10)
    public void testSiteLogo() {
        // Поиск логотипа по различным селекторам
        WebElement logo = null;
        try {
            logo = driver.findElement(By.cssSelector(".brand-logo, .logo, img[alt*='logo'], .jdi-logo"));
        } catch (Exception e) {
            // Если не нашли по классам, ищем первый img в header
            logo = driver.findElement(By.cssSelector("header img"));
        }

        assertNotNull(logo, "Логотип сайта не найден");
        assertTrue(logo.isDisplayed(), "Логотип не отображается");

        // Проверка alt атрибута
        String alt = logo.getAttribute("alt");
```

```
        assertNotNull(alt, "У логотипа отсутствует атрибут alt");

        // Проверка кликабельности логотипа (должен вести на главную)
        WebElement logoLink = logo.findElement(By.xpath("./ancestor::a[1]"));
        assertNotNull(logoLink, "Логотип не обернут в ссылку");

        String href = logoLink.getAttribute("href");
        assertNotNull(href, "У ссылки логотипа отсутствует href");
        assertTrue(href.contains("index") || href.endsWith("/") || href.contains
            ("jdi-testing"),
                "Логотип должен вести на главную страницу");
    }

    // 9. Проверка футера страницы
    @Test(priority = 11)
    public void testFooterContent() {
        // Проверяем наличие футера (не обязательно тега <footer>, может быть
            div/class)
        WebElement footer = driver.findElement(By.cssSelector("footer, .footer,
            [role='contentinfo']"));
        assertTrue(footer.isDisplayed(), "Футер страницы не отображается");

        // Проверка, что футер содержит какой-либо текст
        String footerText = footer.getText().trim();
        assertFalse(footerText.isEmpty(), "Текст футера пустой");

        // Опциональная проверка: футер должен содержать хотя бы одно из ожидаем
            ых слов
        // (если структура сайта изменится, тест не упадёт критично)
        boolean hasExpectedContent =
                footerText.toLowerCase().contains("o") ||
                        footerText.toLowerCase().contains("copyright") ||
                        footerText.matches(".*\\d{4}.*") ||
                        footerText.toLowerCase().contains("jdi") ||
                        footerText.toLowerCase().contains("epam");

        assertTrue(hasExpectedContent,
                "Футер не содержит ожидаемого контента. Текст: '" + footerText +
                    "'");
    }

    // 10. Проверка responsive элементов (наличие классов адаптивности)
    @Test(priority = 12)
    public void testResponsiveClasses() {
        // Проверка наличия container или wrapper классов
        WebElement mainContainer = driver.findElement(By.cssSelector(".container
            , .wrapper, .main-content"));
        assertNotNull(mainContainer, "Отсутствует основной контейнер страницы");

        // Проверка, что элементы имеют классы для адаптивности
        String containerClass = mainContainer.getAttribute("class");
        assertTrue(containerClass.contains("container") ||
                        containerClass.contains("wrapper") ||
                        containerClass.contains("content"),
                "Основной контейнер должен иметь класс container/wrapper/content
                    ");
    }
}
```